Clustering Business Case

December 19, 2025

0.1 Introduction

- Scaler, as an emerging tech-versity, endeavors to provide world-class education in computer science & data science domains
- A significant challenge for Scaler is understanding the diverse backgrounds of its learners, especially in terms of their current roles, companies, and experience Clustering similar learners helps in customizing the learning experience, there y increasing retention and satisfact.
- Analyzing the vast data of learners can uncover patterns in their professional backgrounds and preferences. This allows Scaler to make tailored conten recommendations and provide specialized mentorshi
- By leveraging data science and unsupervised learning, particularly clustering techniques, Scaler can group learners with similar profiles, aiding in delivering more personalized learning journey

0.1.1 What is Expected?

As a data scientist at Scaler, you are expected to:

Data Understanding & Preparation

- Explore and clean the dataset related to companies, job roles, learner profiles, and outcomes.
- Handle missing values, outliers, and perform necessary feature engineering.

Clustering Analysis

- Apply appropriate clustering techniques (e.g., K-Means, Hierarchical Clustering, DBSCAN) to group companies and job positions based on relevant attributes.
- Justify the choice of clustering algorithms and feature selection.

Cluster Evaluation

- Evaluate the coherence and quality of clusters using suitable metrics such as Silhouette Score, Elbow Method, or Davies–Bouldin Index.
- Interpret cluster characteristics and ensure business relevance.

Insights & Interpretation

- Profile the best-performing companies and job roles within each cluster.
- Identify patterns related to learner outcomes, placement success, or skill demand.

Actionable Recommendations

- Provide data-driven insights to improve learner profiling.
- Suggest how Scaler can tailor courses, curricula, or career guidance based on cluster insights.
- Highlight potential implications for reducing learner churn and improving placement outcomes.

Communication

- Present findings clearly using visualizations and concise explanations.
- Summarize key takeaways for both technical and non-technical stakeholders.

0.2 1. Data

The analysis was done on the data located at - https://d2beiqkhq929f0.cloudfront.net/public_assets/assets/000/00

0.3 2. Libraries

Below are the libraries required

[ ]:
```
!pip install umap-learn
```

[5]:
```
# libraries to analyze data
import numpy as np
import pandas as pd

# libraries to visualize data
import matplotlib.pyplot as plt
import seaborn as sns

from random import sample
from numpy.random import uniform

import re

from sklearn.preprocessing import LabelEncoder, OneHotEncoder, StandardScaler
from sklearn.impute import KNNImputer
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
```

```
import scipy.cluster.hierarchy as sch
from scipy.cluster.hierarchy import dendrogram

import umap
```

0.4 3. Data Loading

Loading the data into Pandas dataframe for easily handling of data

[6]:
```
# read the file into a pandas dataframe
df = pd.read_csv('scaler_clustering.csv')
# look at the datatypes of the columns
print('*************************************************')
print(df.info())
print('*************************************************\n')
print('*************************************************')
print(f'Shape of the dataset is {df.shape}')
print('*************************************************\n')
print('*************************************************')
print(f'Number of nan/null values in each column: \n{df.isna().sum()}')
print('*************************************************\n')
print('*************************************************')
print(f'Number of unique values in each column: \n{df.nunique()}')
print('*************************************************\n')
print('*************************************************')
print(f'Duplicate entries: \n{df.duplicated().value_counts()}')
```

```
*************************************************
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205843 entries, 0 to 205842
Data columns (total 7 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Unnamed: 0        205843 non-null  int64
 1   company_hash      205799 non-null  object
 2   email_hash        205843 non-null  object
 3   orgyear           205757 non-null  float64
 4   ctc               205843 non-null  int64
 5   job_position      153279 non-null  object
 6   ctc_updated_year  205843 non-null  float64
dtypes: float64(2), int64(2), object(3)
memory usage: 11.0+ MB
None
*************************************************

*************************************************
Shape of the dataset is (205843, 7)
*************************************************
```

```
************************************************
Number of nan/null values in each column:
Unnamed: 0              0
company_hash           44
email_hash              0
orgyear                86
ctc                     0
job_position        52564
ctc_updated_year        0
dtype: int64
************************************************

************************************************
Number of unique values in each column:
Unnamed: 0          205843
company_hash         37299
email_hash          153443
orgyear                 77
ctc                   3360
job_position          1016
ctc_updated_year         7
dtype: int64
************************************************

************************************************
Duplicate entries:
False    205843
Name: count, dtype: int64
```

[7]:
```
# look at the top 5 rows
df.head(5)
```

[7]:

	Unnamed: 0	company_hash	email_hash	orgyear	ctc
0	0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05…	2016.0	1100000
1	1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10…	2018.0	449999
2	2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9…	2015.0	2000000
3	3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58…	2017.0	700000
4	4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520…	2017.0	1400000

```
         job_position  ctc_updated_year
0               Other            2020.0
1  FullStack Engineer            2019.0
2    Backend Engineer            2020.0
3    Backend Engineer            2019.0
4  FullStack Engineer            2019.0
```

[8]:
```
df.describe()
```

[8]:
```
          Unnamed: 0        orgyear           ctc  ctc_updated_year
count  205843.000000  205757.000000  2.058430e+05     205843.000000
mean   103273.941786    2014.882750  2.271685e+06       2019.628231
std     59741.306484      63.571115  1.180091e+07          1.325104
min         0.000000       0.000000  2.000000e+00       2015.000000
25%     51518.500000    2013.000000  5.300000e+05       2019.000000
50%    103151.000000    2016.000000  9.500000e+05       2020.000000
75%    154992.500000    2018.000000  1.700000e+06       2021.000000
max    206922.000000   20165.000000  1.000150e+09       2021.000000
```

[9]:
```
df.describe(include='object')
```

[9]:
```
                    company_hash  \
count                     205799
unique                     37299
top     nvnv wgzohrnvzwj otqcxwto
freq                        8337

                                                email_hash      job_position
count                                               205843            153279
unique                                              153443              1016
top     bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…  Backend Engineer
freq                                                    10             43554
```

Insight

- There are 205843 entries with 7 columns
- There are 44 null/missing values in company_hash, 86 in orgyear and 52564 in job_position
- There are no duplicates
- There are 1016 unique job_position
- The column Unnamed: 0 can be dropped as it doesnt provide any new information

[10]:
```
# Drop "Unnamed: 0" column
df.drop(columns=['Unnamed: 0'], inplace=True)

def preprocess_string(string):
    new_string= re.sub('[^A-Za-z ]+', '', string).lower().strip()
```

```
    return new_string

# Normalize the string
df["company_hash"] = df["company_hash"].apply(lambda x:␣
 ↪preprocess_string(str(x)))
df["job_position"] = df["job_position"].apply(lambda x:␣
 ↪preprocess_string(str(x)))

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 205843 entries, 0 to 205842
Data columns (total 6 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   company_hash      205843 non-null  object
 1   email_hash        205843 non-null  object
 2   orgyear           205757 non-null  float64
 3   ctc               205843 non-null  int64
 4   job_position      205843 non-null  object
 5   ctc_updated_year  205843 non-null  float64
dtypes: float64(2), int64(1), object(3)
memory usage: 9.4+ MB
```

[11]:
```
# look at the top 10 rows
df.head(10)
```

[11]:

	company_hash	email_hash	orgyear	ctc	ctc_updated_year
0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05…	2016.0	1100000	2020.0
1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10…	2018.0	449999	2019.0
2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9…	2015.0	2000000	2020.0
3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58…	2017.0	700000	2019.0
4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520…	2017.0	1400000	2020.0
5	yvuuxrj hzbvqqxta bvqptnxzs ucn rna	18f2c4aa2ac9dd3ae8ff74f32d30413f5165565b90d8f2…	2018.0	700000	2019.0
6	lubgqsvz wyvot wg	9bf128ae3f4ea26c7a38b9cdc58cf2acbb8592100c4128…	2018.0	1500000	2020.0
7	vwwtznhqt ntwyzgrgsj				
8	utqoxontzn ojontbo				
9	xrbhd				

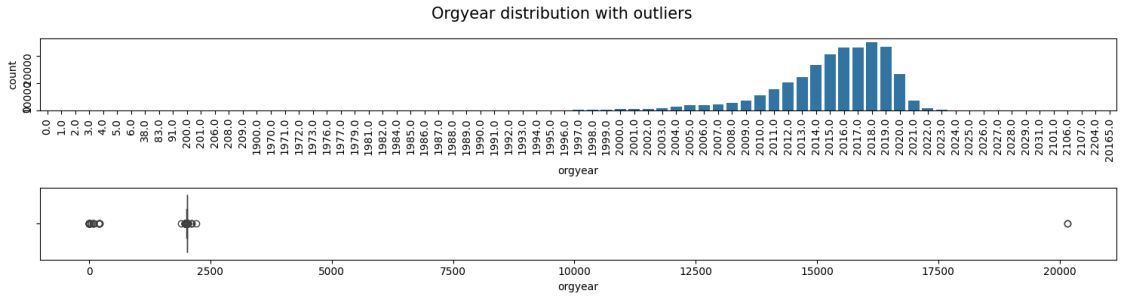
```
7  756d35a7f6bb8ffeaffc8fcca9ddbb78e7450fa0de2be0…   2019.0   400000
8  e245da546bf50eba09cb7c9976926bd56557d1ac9a17fb…   2020.0   450000
9  b2dc928f4c22a9860b4a427efb8ab761e1ce0015fba1a5…   2019.0   360000
```

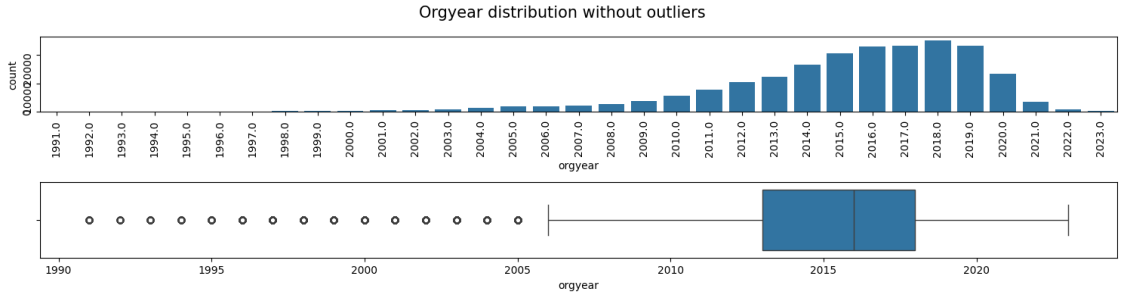
	job_position	ctc_updated_year
0	other	2020.0
1	fullstack engineer	2019.0
2	backend engineer	2020.0
3	backend engineer	2019.0
4	fullstack engineer	2019.0
5	fullstack engineer	2020.0
6	fullstack engineer	2019.0
7	backend engineer	2019.0
8	nan	2019.0
9	nan	2019.0

0.5 Exploratory Data Analysis

0.5.1 Univariate analysis

[12]:
```
data = df['orgyear']
fig, axs = plt.subplots(2,1,figsize=(15,4))
sns.countplot(ax = axs[0], x=data)
axs[0].tick_params(labelrotation=90)
sns.boxplot(ax = axs[1], x=data)
fig.suptitle('Orgyear distribution with outliers', fontsize=15)
plt.tight_layout()
plt.show()

lower_bound = df['orgyear'].quantile(0.001)
upper_bound = df['orgyear'].quantile(0.999)
data = data[(data >= lower_bound) & (data <= upper_bound)]
fig, axs = plt.subplots(2,1,figsize=(15,4))
sns.countplot(ax = axs[0], x=data)
axs[0].tick_params(labelrotation=90)
sns.boxplot(ax = axs[1], x=data)
fig.suptitle('Orgyear distribution without outliers', fontsize=15)
plt.tight_layout()
plt.show()
```



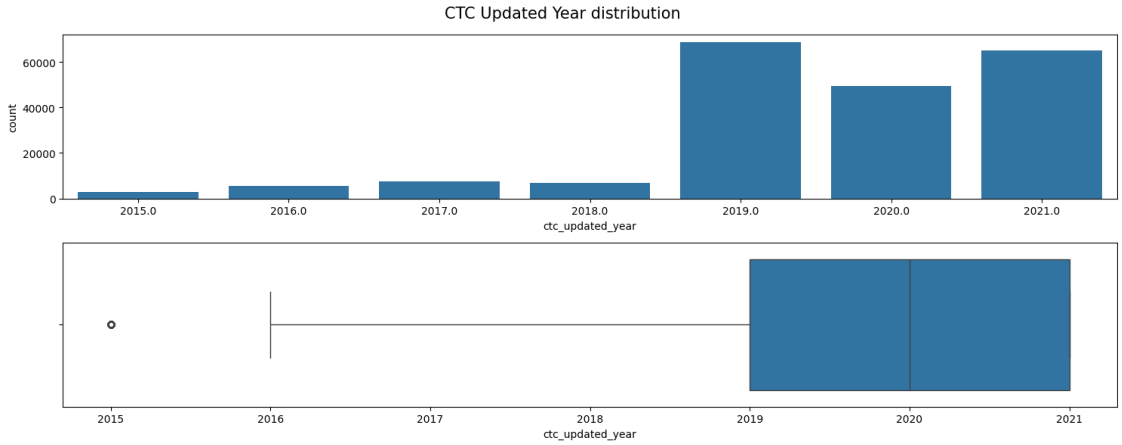


Insight

- The column orgyear has a lot of errors. The years close to 0 and the years greater than the current year are all outliers
- Maximum number of learners began their employment at the current company in the year 2018
- The distribution is left skewed, which is obvious as there are learners who have been working from long time too

[13]:
```
df = df[(df['orgyear'] >= lower_bound) & (df['orgyear'] <= upper_bound)]
df.reset_index(drop=True, inplace=True)
```

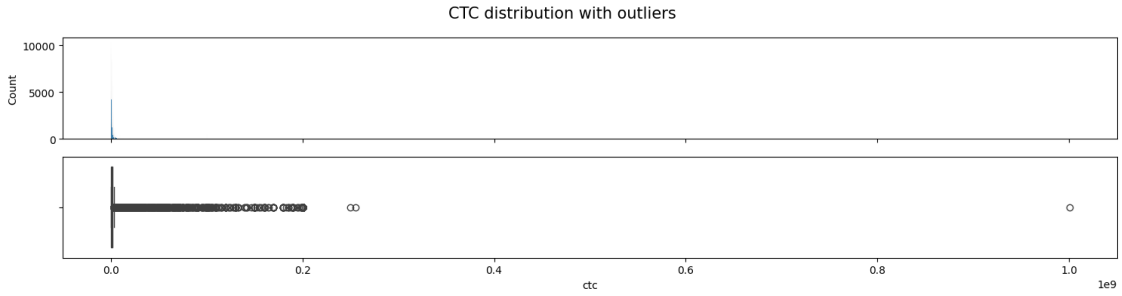
[14]:
```
data = df['ctc_updated_year']
fig, axs = plt.subplots(2,1,figsize=(15,6))
sns.countplot(ax = axs[0], x=data)
sns.boxplot(ax = axs[1], x=data)
fig.suptitle('CTC Updated Year distribution', fontsize=15)
plt.tight_layout()
plt.show()
```

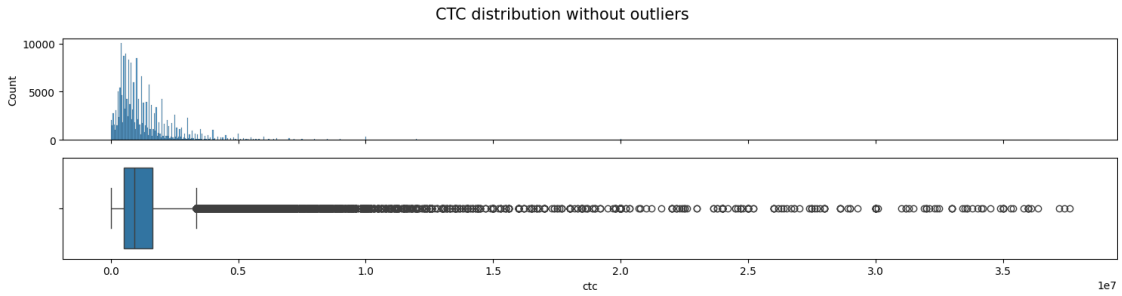


Insight

- Maximum learners got their CTC updated in the year 2019, 2020 and 2021

[15]:
```
data = df['ctc']
fig, axs = plt.subplots(2,1,figsize=(15,4), sharex=True)
sns.histplot(ax = axs[0], x=data)
sns.boxplot(ax = axs[1], x=data)
fig.suptitle('CTC distribution with outliers', fontsize=15)
plt.tight_layout()
plt.show()

mean = data.mean()
std = data.std()
lower_bound = mean - (3*std)
upper_bound = mean + (3*std)
data = data[(data > lower_bound) & (data < upper_bound)]
fig, axs = plt.subplots(2,1,figsize=(15,4), sharex=True)
sns.histplot(ax = axs[0], x=data)
sns.boxplot(ax = axs[1], x=data)
fig.suptitle('CTC distribution without outliers', fontsize=15)
plt.tight_layout()
plt.show()
```





Insight

- The distribution of CTC is extremely right skewed with an obvious outlier being at CTC around 1.0E9
- Without the outlier also, the CTC is right skewed as there are good number of learners with higher CTC

[16]:
```
df = df[(df['ctc'] >= lower_bound) & (df['ctc'] <= upper_bound)]
df.reset_index(drop=True, inplace=True)
```

[17]:
```
df['company_hash'].value_counts()[:10]
```

[17]:
```
company_hash
nvnv wgzohrnvzwj otqcxwto    8266
xzegojo                      5348
vbvkgz                       3446
zgn vuurxwvmrt vwwghzn       3356
wgszxkvzn                    3212
vwwtznhqt                    2833
fxuqg rxbxnta                2622
gqvwrt                       2496
bxwqgogen                    2121
wvustbxzx                    2026
Name: count, dtype: int64
```

Insight

- Maximum number of learners have their current employer whose company hash is nvnv wgzohrnvzwj otqcxwto

[18]:
```
df['email_hash'].value_counts()[:10]
```

[18]:
```
email_hash
bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b    10
```

```
3e5e49daa5527a6d5a33599b238bf9bf31e85b9efa9a94f1c88c5e15a6f31378     9
6842660273f70e9aa239026ba33bfe82275d6ab0d20124021b952b5bc3d07e6c     9
298528ce3160cc761e4dc37a07337ee2e0589df251d73645aae209b010210eee     9
c0eb129061675da412b0deb15871dd06ef0d7cd86eb5f7e8cc6a20b0d1938183     8
d15041f58bb01c8ee29f72e33b136e26bc32f3169a40b53d75fe7ae9cbb9a551     8
faf40195f8c58d5c7edc758cc725a762d51920da996410b80ac4a4d85c803da0     8
4818edfd67ed8563dde5d083306485d91d19f4f1c95d193a1700e79dd245b75c     8
d598d6f1fb21b45593c2afc1c2f76ae9f4cb7167156cdf93246d4192a89d8065     8
b4d5afa09bec8689017d8b29701b80d664ca37b83cb883376b2e95191320da66     8
Name: count, dtype: int64
```

Insight

- It is suprising to see that many learners have the same email id, with maximum(10) learners having email with hash bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b

[19]:
```
df['job_position'].value_counts()
```

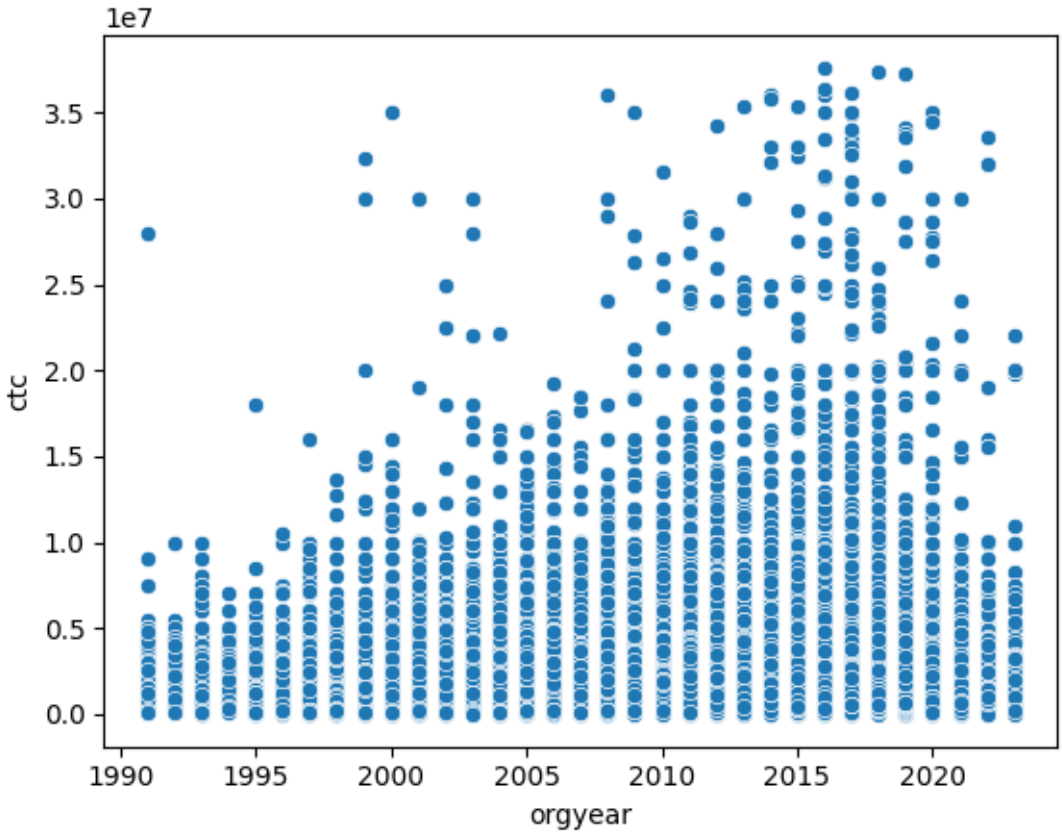
[19]:
```
job_position
nan                             52166
backend engineer                43336
fullstack engineer              25826
other                           17628
frontend engineer               10341
                                …
sr ios developer                    1
jr software engineer                1
product solution engineer           1
java software engineer              1
qaeintern                           1
Name: count, Length: 848, dtype: int64
```

Insight

- Maximum number of learners are Backend Engineers

0.5.2 Bivariate analysis

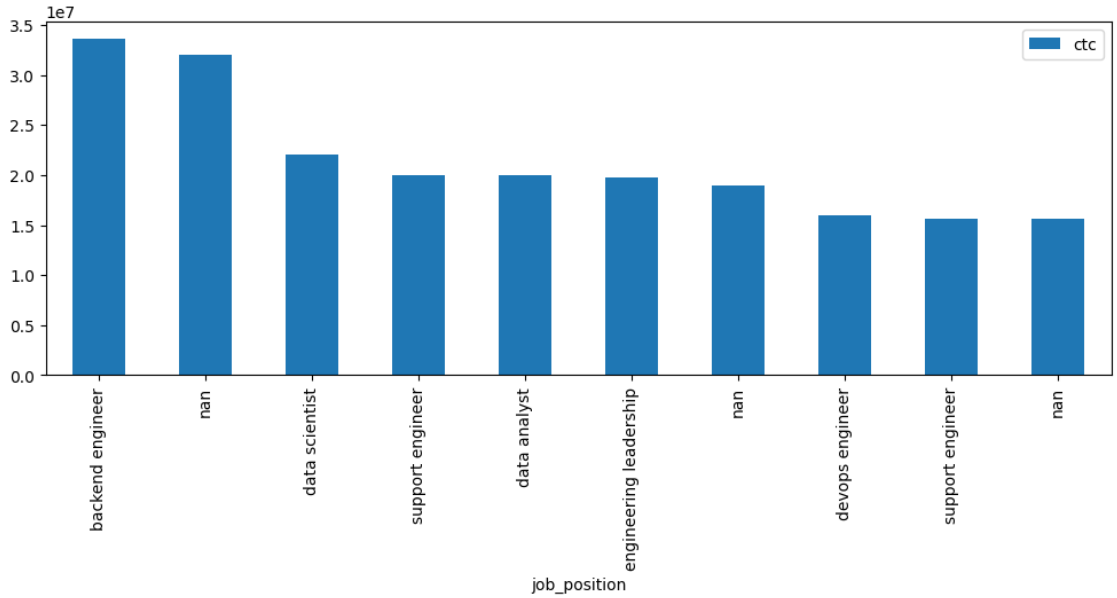
[20]:
```
sns.scatterplot(data=df, x='orgyear', y='ctc')
plt.show()
```



Insight

- It is obvious that the learners who joined/changed company recently have a higher CTC

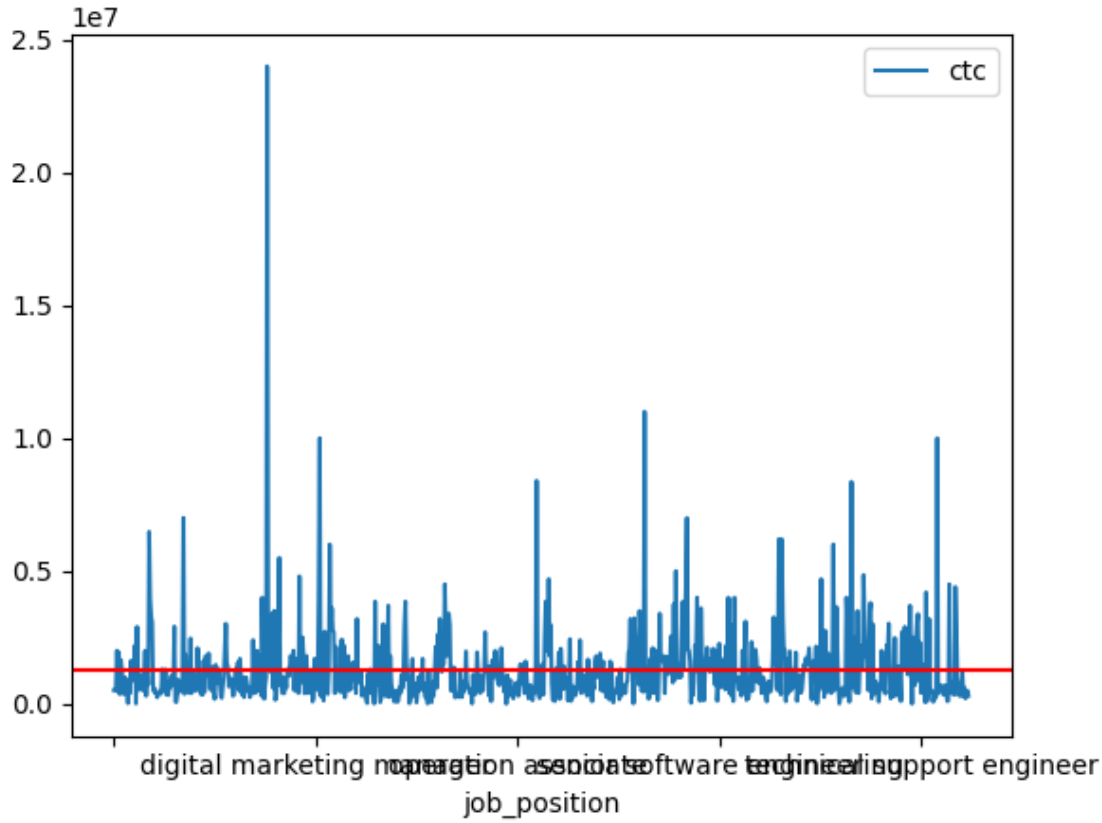
[21]:
```
df[(df['job_position'] != 'other') &
(~df['job_position'].isna()) &
(df['orgyear'] > 2021)][['ctc', 'job_position']].sort_values(by='ctc',␣
 ↪ascending=False).head(10).plot(kind='bar', x = 'job_position', y='ctc',␣
 ↪figsize=(12,4))
plt.show()
```



Insight

- Above plot shows few of the positions with top CTCs
- It has a mix of software developers, data analysts/scients, leadership etc

[22]:
```
temp_df = df.groupby(['job_position']).agg({'ctc':'mean'})
temp_df.plot(kind='line')
ctc_mean = round(temp_df['ctc'].mean(),2)
plt.axhline(y = ctc_mean, color = 'r', linestyle = '-')
plt.show()
print(f'Insight: Average CTC across different job positions is {ctc_mean}')
```

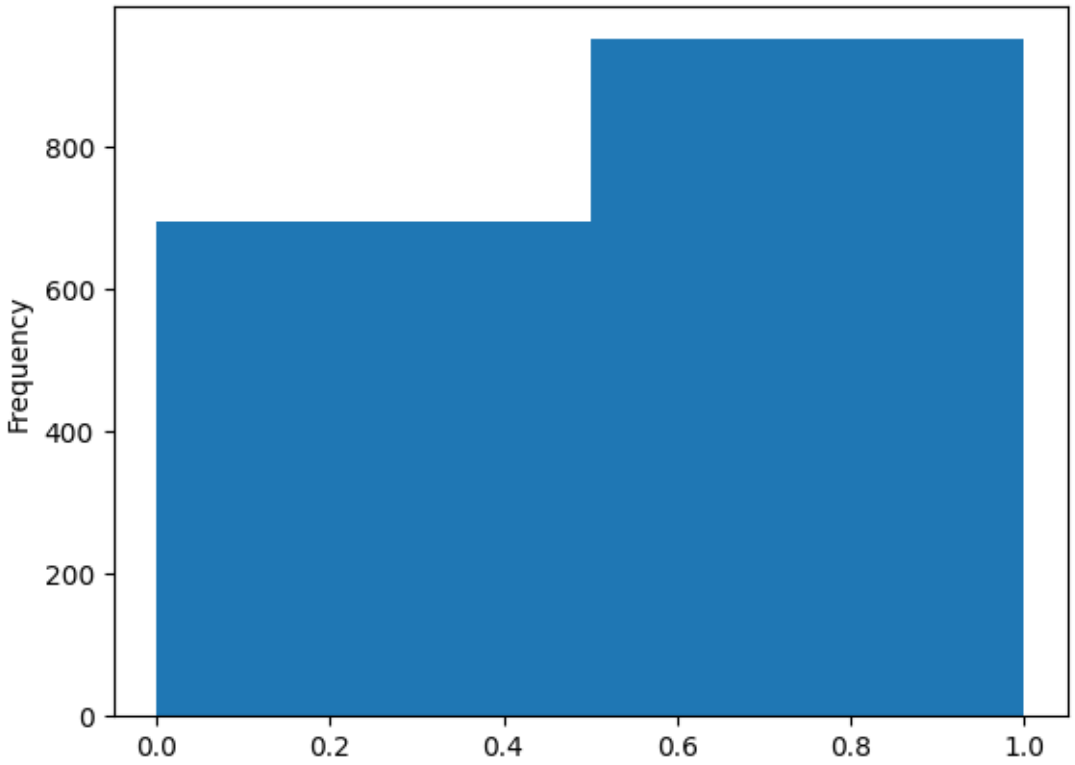


Insight: Average CTC across different job positions is 1268701.92

0.5.3 Multivariate analysis

[23]:
```
temp_df = df[~df['job_position'].isna()]
temp_df = temp_df.groupby(['company_hash', 'job_position']).agg({'ctc':'mean'}).
 ↪reset_index()
temp_df_da = temp_df[[True if ('data' and 'scientist' in x) else False for x in␣
 ↪temp_df['job_position']]]
temp_df_da = temp_df_da.drop(columns=['job_position']).rename(columns={'ctc':
 ↪'ctc_ds'}).reset_index(drop=True)
temp_df_others = temp_df[[True if ('data' and 'scientist' not in x) else False␣
 ↪for x in temp_df['job_position']]]
temp_df_others = temp_df_others.drop(columns=['job_position']).
 ↪rename(columns={'ctc':'ctc_others'}).reset_index(drop=True)
temp_df_common = temp_df_da.merge(temp_df_others, on = 'company_hash',␣
 ↪how='inner')
temp_df_common['ctc_ds_gt_ctc_others'] = temp_df_common['ctc_ds'] >␣
 ↪temp_df_common['ctc_others']
```

```
temp_df_common.groupby(['company_hash']).agg({'ctc_ds_gt_ctc_others':'mean'}).
  ↪reset_index()['ctc_ds_gt_ctc_others'].plot(kind='hist', bins=2)
plt.show()
```



Insight

- There are around 900+ companies in which more than 50% of the times the avergae CTC of a Data Scientist is greater than that of other roles

0.6 Data Preprocessing

[24]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 203956 entries, 0 to 203955
Data columns (total 6 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   company_hash      203956 non-null  object
 1   email_hash        203956 non-null  object
 2   orgyear           203956 non-null  float64
 3   ctc               203956 non-null  int64
 4   job_position      203956 non-null  object
```

```
 5   ctc_updated_year  203956 non-null  float64
dtypes: float64(2), int64(1), object(3)
memory usage: 9.3+ MB
```

0.6.1 Handling duplicate values

[25]:
```
df[df['email_hash'] ==␣
 ↪'bbace3cc586400bbc65765bc6a16b77d8913836cfc98b77c05488f02f5714a4b']
```

[25]:

	company_hash	email_hash	orgyear	ctc	job_position	ctc_updated_year
23498	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	720000	nan	2020.0
45062	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	720000	support engineer	2020.0
71181	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	720000	other	2020.0
101498	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	720000	fullstack engineer	2020.0
116226	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	720000	data analyst	2020.0
119910	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	660000	other	2019.0
122888	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	660000	support engineer	2019.0
142804	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	660000	fullstack engineer	2019.0
151096	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	660000	devops engineer	2019.0
158105	oxej ntwyzgrgsxto rxbxnta	bbace3cc586400bbc65765bc6a16b77d8913836cfc98b7…	2018.0	660000	nan	2019.0

There should be a unique entry for a combination of employee's e-mail and CTC. I will remove all the duplicates by keeping only the first entry as the first entry seems to be the latest entry

[26]:
```
df = df.drop_duplicates(subset=['email_hash', 'ctc'])
df.reset_index(drop=True, inplace=True)
```

[27]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 162004 entries, 0 to 162003
Data columns (total 6 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   company_hash      162004 non-null  object
 1   email_hash        162004 non-null  object
 2   orgyear           162004 non-null  float64
 3   ctc               162004 non-null  int64
 4   job_position      162004 non-null  object
 5   ctc_updated_year  162004 non-null  float64
dtypes: float64(2), int64(1), object(3)
memory usage: 7.4+ MB
```

[28]:
```
df['email_hash'].value_counts()[:10]
```

[28]:
```
email_hash
ee309ce613f9a0b6d1e210c631cb922dedf89480d39603ff94e08376cdcf3b79    2
3e4374500ad5138018444d49fe035e45719e9922863f491fdda9dc40f6a0b74a    2
92c3ccfaf557db3d4ded734c9e06b836cb64a786ebafe04b8d6bff969215f2da    2
72c2171a022115d475c8faac306912a4c95f6dd7fdd320df09a5e9160d2a8385    2
7f24d2f5171ea469482a9966832237bc023678883ecd0c5142677b75a138b2fa    2
b08f5ecd63a5ec14c2143ea00f6f8ce09db420841c902b12b98ca07c56527b81    2
0c018dda4109b925c02ae889a9021568dd66f7fd7a715c73840df1bb8b726acb    2
64e39ac4c4dcf057f003fbcccd4d38d6de63beacdc7aef98b1d1e09add22d586    2
830c44b20dd9e8d2843e504b15bd9dace5e4cd86cee1f7882d2005dad91af7a1    2
65e5f572be43c76874d58104b741df640f2048216259892838b5b4107c6afad0    2
Name: count, dtype: int64
```

[29]:
```
df[df['email_hash'] ==␣
  ↪'bd126a265ff5985a859b2226e38d99eaa786771b2f3e0564adccef4772964497']
```

[29]:
```
      company_hash                                         email_hash  \
77199       vbvkgz  bd126a265ff5985a859b2226e38d99eaa786771b2f3e05…
78210       vbvkgz  bd126a265ff5985a859b2226e38d99eaa786771b2f3e05…

       orgyear     ctc job_position  ctc_updated_year
77199   2018.0  320000          nan            2020.0
78210   2018.0  235999        other            2019.0
```

0.6.2 Handling null values

[30]:
```
df.isna().sum()
```

[30]:
```
company_hash        0
email_hash          0
orgyear             0
ctc                 0
job_position        0
ctc_updated_year    0
dtype: int64
```

[31]:
```
print(f'Number of empty company hash is {(df["company_hash"] == "").sum()}')
print(f'Number of company hash with "nan" values is {(df["company_hash"] ==␣
 ↪"nan").sum()}')
print(f'Number of empty job position is {(df["job_position"] == "").sum()}')
print(f'Number of job position with "nan" values is {(df["job_position"] ==␣
 ↪"nan").sum()}')
```

```
Number of empty company hash is 64
Number of company hash with "nan" values is 37
Number of empty job position is 6
Number of job position with "nan" values is 35841
```

Insight

- I will remove records where company hash is empty or “nan”
- I will remove records where job position is empty
- I will use imputation for job position with “nan” values

[32]:
```
df = df[~((df["company_hash"] == "") | (df["company_hash"] == "nan") |␣
 ↪(df["job_position"] == ""))]
df.reset_index(drop=True, inplace=True)

print(f'Number of empty company hash is {(df["company_hash"] == "").sum()}')
print(f'Number of company hash with "nan" values is {(df["company_hash"] ==␣
 ↪"nan").sum()}')
print(f'Number of empty job position is {(df["job_position"] == "").sum()}')
print(f'Number of job position with "nan" values is {(df["job_position"] ==␣
 ↪"nan").sum()}')

df.loc[df['job_position']=='nan', 'job_position']=np.nan
df.isna().sum()
```

```
Number of empty company hash is 0
Number of company hash with "nan" values is 0
Number of empty job position is 0
Number of job position with "nan" values is 35787
```

[32]:
```
company_hash            0
email_hash              0
orgyear                 0
ctc                     0
job_position        35787
ctc_updated_year        0
dtype: int64
```

[33]:
```
temp_df = df[['company_hash', 'orgyear', 'ctc', 'job_position',␣
 ↪'ctc_updated_year']].copy()
encoders = dict()
columns_to_encode = ['company_hash', 'job_position']
for col in columns_to_encode:
    series = temp_df[col]
    encoder = LabelEncoder()
    temp_df[col] = pd.Series(encoder.fit_transform(series[series.notnull()]),
                             index = series[series.notnull()].index)
    encoders[col] = encoder

imputer = KNNImputer(n_neighbors=1)
temp_df = pd.DataFrame(imputer.fit_transform(temp_df), columns=temp_df.columns)
temp_df['job_position'] = encoders['job_position'].
 ↪inverse_transform(temp_df['job_position'].astype('int32'))
```

[34]:
```
df['job_position'] = temp_df['job_position']

print(f'Number of empty company hash is {(df["company_hash"] == "").sum()}')
print(f'Number of company hash with "nan" values is {(df["company_hash"] ==␣
 ↪"nan").sum()}')
print(f'Number of empty job position is {(df["job_position"] == "").sum()}')
print(f'Number of job position with "nan" values is {(df["job_position"] ==␣
 ↪"nan").sum()}')

df.isna().sum()
```

```
Number of empty company hash is 0
Number of company hash with "nan" values is 0
Number of empty job position is 0
Number of job position with "nan" values is 0
```

[34]:
```
company_hash        0
email_hash          0
orgyear             0
ctc                 0
job_position        0
ctc_updated_year    0
dtype: int64
```

0.6.3 Outlier Treatment

Outlier have already been taken care of

0.6.4 Feature Engineering

Handle entries having ctc_updated_year less than orgyear

[35]:
```
value = (df['orgyear'] > df['ctc_updated_year']).sum()
print(f'Before -> Number of entries where ctc_updated_year is less than orgyear:
  ↪ {value}')
df['ctc_updated_year'] = df[["ctc_updated_year","orgyear"]].max(axis = 1)
value = (df['orgyear'] > df['ctc_updated_year']).sum()
print(f'After -> Number of entries where ctc_updated_year is less than orgyear:␣
  ↪{value}')
```

```
Before -> Number of entries where ctc_updated_year is less than orgyear: 7331
After -> Number of entries where ctc_updated_year is less than orgyear: 0
```

Extract years of experience and years since increment values

[36]:
```
current_year = 2023
df['years_of_experience'] = current_year - df['orgyear']
df['years_since_increment'] = current_year - df['ctc_updated_year']
```

Calculate CTC rank by comparing the CTC seperately against average CTC per company, average CTC per job position and average CTC per years of experience

- Value 1 - CTC is greater than all three average CTCs
- Value 2 - CTC is greater than at least 2 of the average CTCs
- Value 3 - CTC is greater than at least 1 of average CTCs
- Value 4 - CTC is less than all the three average CTCs

[37]:
```
df.head()
```

[37]:

	company_hash	email_hash	orgyear	ctc
0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05…	2016.0	1100000
1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10…	2018.0	449999
2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9…	2015.0	2000000
3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58…	2017.0	700000
4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520…	2017.0	1400000

```
         job_position  ctc_updated_year  years_of_experience  \
0               other            2020.0                  7.0
1  fullstack engineer            2019.0                  5.0
2    backend engineer            2020.0                  8.0
3    backend engineer            2019.0                  6.0
4  fullstack engineer            2019.0                  6.0

   years_since_increment
0                    3.0
1                    4.0
2                    3.0
3                    4.0
4                    4.0
```

[38]:
```
df1 = df.groupby(['company_hash']).agg({'ctc':'mean'}).reset_index().
 ↪rename(columns = {'ctc':'avg_ctc_per_C'})
df2 = df.groupby(['job_position']).agg({'ctc':'mean'}).reset_index().
 ↪rename(columns = {'ctc':'avg_ctc_per_J'})
df3 = df.groupby(['years_of_experience']).agg({'ctc':'mean'}).reset_index().
 ↪rename(columns = {'ctc':'avg_ctc_per_E'})
df = df.merge(df1, on='company_hash', how='left')
df = df.merge(df2, on='job_position', how='left')
df = df.merge(df3, on='years_of_experience', how='left')
def calculate_ctc_rnk(ctc, acpc, acpj, acpe):
    if ctc > acpc and ctc > acpj and ctc > acpe:
        return 1
    elif (ctc > acpc and ctc > acpj) or (ctc > acpc and ctc > acpe) or (ctc >␣
 ↪acpj and ctc > acpe):
        return 2
    elif ctc > acpc or ctc > acpj or ctc > acpe:
        return 3
    else:
        return 4
df['ctc_rnk'] = df.apply(lambda x: calculate_ctc_rnk(x['ctc'],
                                                     x['avg_ctc_per_C'],
                                                     x['avg_ctc_per_J'],
                                                     x['avg_ctc_per_E']),
                         axis=1)
df.drop(columns=['avg_ctc_per_C', 'avg_ctc_per_J', 'avg_ctc_per_E'],␣
 ↪inplace=True)
```

Calculate designation, class and tier values based on CTC statistics on company-experience level, company-position level and company level

- Value 1 - CTC is greater than 75% of the population of the group
- Value 2 - CTC is between 50% and 75% of the population of the group

- Value 2 - CTC is less than 50% of the population of the group

[39]:
```
def group_ctc(x,x50,x75):
    if x < x50:
        return 3
    elif x >= x50 and x <= x75:
        return 2
    elif  x > x75:
        return 1
```

[40]:
```
temp_df_CE = df.groupby(['company_hash', 'years_of_experience'])['ctc'].
 ↪describe()
df_CE = df.merge(temp_df_CE, on=['company_hash', 'years_of_experience'],␣
 ↪how='left')
temp_df_CJ = df.groupby(['company_hash', 'job_position'])['ctc'].describe()
df_CJ = df.merge(temp_df_CJ, on=['company_hash', 'job_position'], how='left')
temp_df_C = df.groupby(['company_hash'])['ctc'].describe()
df_C = df.merge(temp_df_C, on=['company_hash'], how='left')
```

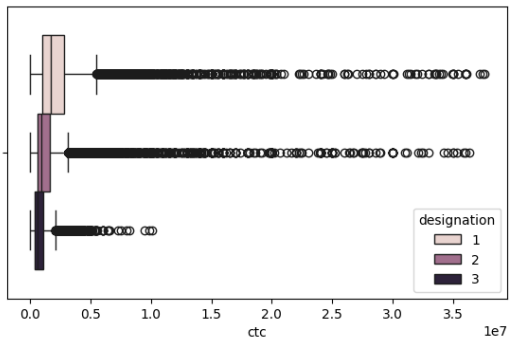
[41]:
```
df['designation'] = df_CE.apply(lambda x: group_ctc(x['ctc'], x["50%"],␣
 ↪x["75%"]), axis=1)
df['class'] = df_CJ.apply(lambda x: group_ctc(x['ctc'], x["50%"], x["75%"]),␣
 ↪axis=1)
df['tier'] = df_C.apply(lambda x: group_ctc(x['ctc'], x["50%"], x["75%"]),␣
 ↪axis=1)
df.head()
```

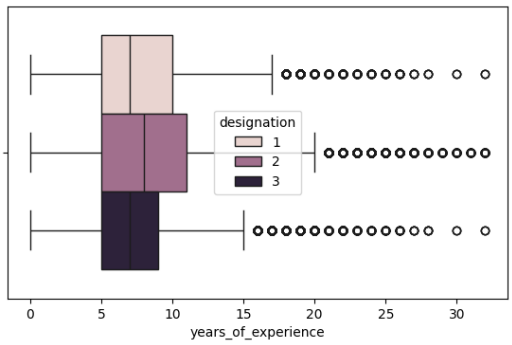
[41]:

	company_hash	email_hash	orgyear	ctc	job_position	ctc_updated_year	years_of_experience
0	atrgxnnt xzaxv	6de0a4417d18ab14334c3f43397fc13b30c35149d70c05…	2016.0	1100000	other	2020.0	7.0
1	qtrxvzwt xzegwgbb rxbxnta	b0aaf1ac138b53cb6e039ba2c3d6604a250d02d5145c10…	2018.0	449999	fullstack engineer	2019.0	5.0
2	ojzwnvwnxw vx	4860c670bcd48fb96c02a4b0ae3608ae6fdd98176112e9…	2015.0	2000000	backend engineer	2020.0	8.0
3	ngpgutaxv	effdede7a2e7c2af664c8a31d9346385016128d66bbc58…	2017.0	700000	backend engineer	2019.0	6.0
4	qxen sqghu	6ff54e709262f55cb999a1c1db8436cb2055d8f79ab520…	2017.0	1400000	fullstack engineer	2019.0	6.0

	years_since_increment	ctc_rnk	designation	class	tier
0	3.0	3	2	1	2
1	4.0	4	3	3	3
2	3.0	2	2	2	2
3	4.0	4	3	3	3
4	4.0	1	2	1	1

[42]:
```
fig, axs = plt.subplots(1,2,figsize=(15,4))
sns.boxplot(ax=axs[0], data=df, x='ctc', hue='designation')
sns.boxplot(ax=axs[1], data=df, x='years_of_experience', hue='designation')
plt.show()
position_counts = df.groupby(['designation', 'job_position']).size().
 ↪reset_index(name='count')
top_positions = (
    position_counts
    .groupby('designation')[['job_position', 'count']]
    .apply(lambda x: x.nlargest(4, 'count'))
    .reset_index()
)

print(top_positions)
```





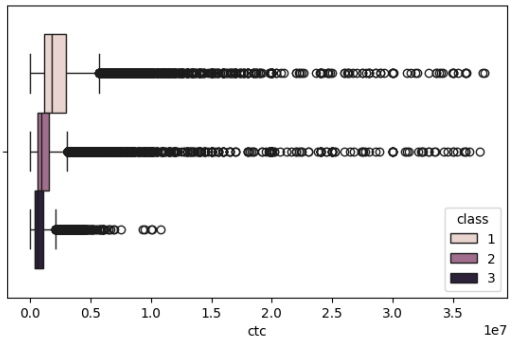
	designation	level_1	job_position	count
0	1	26	backend engineer	10626
1	1	53	fullstack engineer	5429
2	1	75	other	2975
3	1	50	frontend engineer	1861
4	2	223	backend engineer	20944
5	2	293	fullstack engineer	13103
6	2	355	other	8875
7	2	289	frontend engineer	5869
8	3	618	backend engineer	14724
9	3	667	fullstack engineer	7699
10	3	718	other	7443

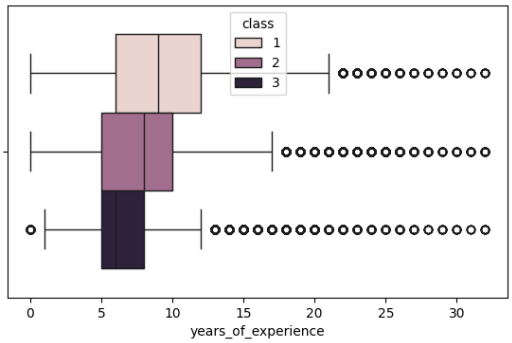
```
11              3      666    frontend engineer   3289
```

Insight

- The mean CTC of designation $1 > 2 > 3$
- The mean years of experience of designation $2 > 1 \sim= 3$
- The top 4 position of all the designations are backend engineer, fullstack engineer, other and frontend engineer
- The clustering based on designation is able to differentiate between CTCs but not years of experience or job position

[43]:
```
fig, axs = plt.subplots(1,2,figsize=(15,4))
sns.boxplot(ax=axs[0], data=df, x='ctc', hue='class')
sns.boxplot(ax=axs[1], data=df, x='years_of_experience', hue='class')
plt.show()
position_counts = df.groupby(['class', 'job_position']).size().
 ↪reset_index(name='count')
top_positions = (
    position_counts
    .groupby('class')[['job_position', 'count']]
    .apply(lambda x: x.nlargest(4, 'count'))
    .reset_index()
)

print(top_positions)
```





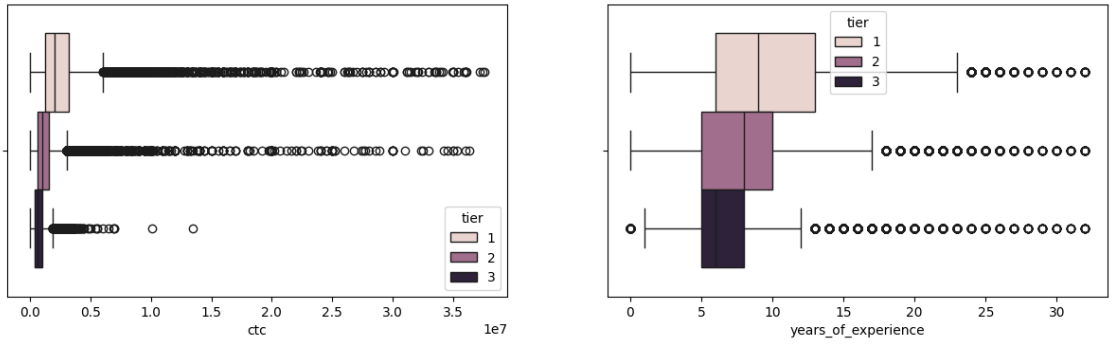
	class	level_1	job_position	count
0	1	10	backend engineer	10049
1	1	22	fullstack engineer	5208
2	1	32	other	3682
3	1	21	frontend engineer	1864
4	2	163	backend engineer	17983
5	2	270	fullstack engineer	12130
6	2	370	other	9076

```
7       2      266   frontend engineer   6071
8       3      709    backend engineer  18262
9       3      721  fullstack engineer   8893
10      3      730               other   6535
11      3      720   frontend engineer   3084
```

Insight

- The mean CTC of class $1 > 2 > 3$
- The mean years of experience of class $1 > 2 > 3$
- The top 4 position of all the class are backend engineer, fullstack engineer, other and frontend engineer
- The clustering based on class is able to differentiate between CTCs, between years of experience but not job position

[44]:
```
fig, axs = plt.subplots(1,2,figsize=(15,4))
sns.boxplot(ax=axs[0], data=df, x='ctc', hue='tier')
sns.boxplot(ax=axs[1], data=df, x='years_of_experience', hue='tier')
plt.show()
position_counts = df.groupby(['tier', 'job_position']).size().
 ↪reset_index(name='count')
top_positions = (
    position_counts
    .sort_values(['tier', 'count'], ascending=[True, False])
    .groupby('tier')
    .head(4)
    .reset_index(drop=True)
)

print(top_positions)
```




```
    tier           job_position  count
0      1        backend engineer  10522
1      1      fullstack engineer   5217
```

```
2      1  engineering leadership   3868
3      1                   other   3148
4      2        backend engineer  17225
5      2      fullstack engineer  10796
6      2                   other   6988
7      2       frontend engineer   4753
8      3        backend engineer  18547
9      3      fullstack engineer  10218
10     3                   other   9157
11     3       frontend engineer   4190
```

Insight

- The mean CTC of tier $1 > 2 > 3$
- The mean years of experience of tier $1 > 2 > 3$
- The top 4 position of tier 1 are backend engineer, fullstack engineer, engineering leadership and other while for all the other tiers are backend engineer, fullstack engineer, other and frontend engineer
- The clustering based on tier is able to differentiate between CTCs, between years of experience and partially between job position

[45]:
```
temp_df = df[df['tier'] == 1].sort_values(by='ctc',␣
 ↪ascending=False)[['company_hash', 'email_hash', 'ctc', 'job_position',␣
 ↪'years_of_experience']]
temp_df.head(10)
```

[45]:
```
                    company_hash  \
25231                 onvnt onqttn
1652                  mxsmvoptnwgb
12240                      stztojo
57410                         xmb
65885                      ofxssj
66457                   jvygg xzw
16073   oytrr xzaxv bvqptno rxbxnta
49297               zgn vuurxwvmrt
112601                 vuuajzvbxwo
8870     nvnv wgzohrnvzwj otqcxwto

                                                email_hash       ctc  \
25231   fb8410d957fcc2e98cc419e9354dbb3acea2309b0898e2…  37600000
1652    89e0bd3c55896b4b09bb31fa4a736dd6c6d9c3622049e0…  37400000
12240   a6e0d878386ba7ef29d50a698c5037864d3eb2cf4de9cb…  37200000
57410   46dece7d152edae30f51b0ceab430cbc9681fdd0100e72…  36100000
65885   4d287c2dd88a8b8008781f9081581d3cd7a36d0cc88c8d…  36000000
66457   2e67d726c283087bbfaef033b250dc4ea395fa2b2d88cd…  36000000
16073   109ea0d2fea7028684f062eccdc53638489ff2e662ea86…  36000000
```

```
49297   2c5f9aefb73259d3a6df5fe503c48587d8fc61eefacc8e…  36000000
112601  4015c0491e4d40f288ee1e0d5d852997bff5535c03bed6…  35820000
8870    914f81589b8d14a404eeee60384fc8e9260f1023ca6636…  35400000

                job_position  years_of_experience
25231                  other                  7.0
1652                   other                  5.0
12240                  other                  4.0
57410   database administrator                6.0
65885                  other                  9.0
66457         backend engineer                15.0
16073                  other                  7.0
49297                  other                  9.0
112601                 other                  9.0
8870                   other                  8.0
```

[46]:
```
temp_df['job_position'].value_counts()[:10]
```

[46]:
```
job_position
backend engineer          10522
fullstack engineer         5217
engineering leadership     3868
other                      3148
frontend engineer          2076
data scientist             1648
android engineer           1027
qa engineer                 975
devops engineer             960
backend architect           742
Name: count, dtype: int64
```

Insight

- Above are the top 10 employees earning more than most of the employees in the company
- The top employees belong to companies which provide software developer roles

[47]:
```
df[df['tier'] == 3].sort_values(by='ctc', ascending=True).
↪head(10)[['company_hash', 'email_hash', 'ctc']]
```

[47]:
```
                    company_hash  \
107027              xzntqcxtfmxn
93403               xzntqcxtfmxn
90133               xzntqcxtfmxn
145643                        xm
92572    hzxctqoxnj ge fvoyxzsngz
79494                       gjg
134689  nvnv wgzohrnvzwj otqcxwto
```

```
118958                         zvz
150114  nvnv wgzohrnvzwj otqcxwto
31213                           zv

                                                email_hash   ctc
107027  3505b02549ebe2c95840ac6f0a35561a3b4cbe4b79cdb1…     2
93403   f2b58aeed3c074652de2cfd3c0717a5d21d6fbcf342a78…     6
90133   23ad96d6b6f1ecf554a52f6e9b61677c7d73d8a409a143…    14
145643  b8a0bb340583936b5a7923947e9aec21add5ebc50cd60b…    15
92572   f7e5e788676100d7c4146740ada9e2f8974defc01f571d…   200
79494   b995d7a2ae5c6f8497762ce04dc5c04ad6ec734d70802a…   600
134689  80ba0259f9f59034c4927cf3bd38dc9ce2eb60ff18135b…   600
118958  9af3dca6c9d705d8d42585ccfce2627f00e1629130d14e…   600
150114  8625d6d072e12dad0c5748ab010e1d0315736a359e2bb5…  1000
31213   8747d9599e2ba1a8624e8bea834ab7a870c89ccca74204…  1000
```

Insight

- Above are the bottom 10 employees earning less than most of the employees in the company

[48]:
```
temp_df = df[df['tier'] == 3].groupby(['company_hash']).agg({'ctc':'mean',
                                                             'years_of_experience':'mean',
                                                             'ctc_rnk':'mean',
                                                             'designation':'mean',
                                                             'class':'mean'}).
 ↪reset_index()
temp_df.describe()
```

[48]:
```
                ctc  years_of_experience      ctc_rnk  designation  \
count  8.956000e+03          8956.000000  8956.000000  8956.000000
mean   7.456474e+05             7.430224     3.747754     2.303444
std    5.655440e+05             3.170351     0.545628     0.383568
min    1.500000e+01             0.000000     1.500000     1.666667
25%    4.000000e+05             5.166667     4.000000     2.000000
50%    6.250000e+05             7.000000     4.000000     2.000000
75%    9.414534e+05             9.000000     4.000000     2.574495
max    1.350000e+07            32.000000     4.000000     3.000000

             class
count  8956.000000
mean      2.350435
std       0.397926
min       1.666667
25%       2.000000
50%       2.000000
75%       2.666667
max       3.000000
```

[49]:
```
df[((df['class'] == 1) & (df['job_position'] == 'data scientist'))].
↪sort_values(by='ctc', ascending=False).head(10)[['company_hash',␣
↪'email_hash', 'ctc']]
```

[49]:

	company_hash	email_hash	ctc
136197	wxnx	f7b7c771ccdbbca7248002ba83f7a176baa974c2c7bb8f…	24200000
61817	bxwqgogen	599e489c815ba51967965c5d6adefd7a76a99ffaa129bd…	22500000
9238	sggsrt	3e290b892b73283b96293c53e4ce4dce2cc6a22399b95c…	22000000
57337	zvz	80f1ae60373f0ada3b75ce19eb585f8cf112de3cfa6ea7…	20000000
90828	xmb xzaxv uqxcvnt rxbxnta	4beee431866cc493f7cc6689c2f00023683575a29e3174…	20000000
98676	nyghsynfgqpo	aa1d9bece779ff63a54bd6d32452e3f938a0afc6a52c6b…	20000000
21231	xmb	b5dc6ad6d8d8f04312c34285a3c45fd9ffdc73ff3f1205…	20000000
149152	onvnt onqttn	9fdab215a86b0e2f18ee6c3d7653442cd7ad8b9cc4cf91…	18000000
21809	exwg	d1290b7e2d85c75902b863ccc3e4aafdd6e6eb07a10a00…	12000000
103726	sgltp	24d6a653ce21e80d3f03eaab9b7600f4bbb0888cf7bccd…	12000000

Insight

- Above are the top 10 employees of data scientist role earning more than their peers

[50]:
```
df[((df['class'] == 3) & (df['job_position'] == 'data scientist'))].
↪sort_values(by='ctc', ascending=True).head(10)[['company_hash',␣
↪'email_hash', 'ctc']]
```

[50]:

	company_hash
92778	ovbohzs trtwnqgzxwo
10011	srgmvrtast xzntrrxstzwt ge nyxzso
8145	bxyhu wgbbhzxwvnxgz
36359	cgavegzt oyvqta otqcxwto rxbxnta
40873	onhatzn
48692	onvnt onqttn
108429	ovbohzs trtwnqg btwyvzxwo
8770	nvnv wgzohrnvzwj otqcxwto
20257	vqxosrgmvr
25625	sggsrt

	email_hash	ctc
92778	3711220da929dcc0f6ca1f150b31ec7ac9302e8e59b118…	3500
10011	8001bc017fbe95541d23f5780c3edb988b7d9b2225e39e…	4000
8145	690f6fdab1ab7514a6a9325ebd6cfe910dbf12d46b6fde…	4000
36359	c5335f1ab2b6b60e4c7bf52a9dd2f22b87e07208ecbb0e…	4000
40873	bd9c04a574090e05b366a81cdb2f3f565d0c60fa8b1647…	6000
48692	210022464f7fc73ab22c22b9b2fcb8dc4fd8e3fe69ac1a…	6000
108429	e374eea75640881206a21894f69190138c2c0535277dc1…	7000
8770	3175d03fd4618eb293d6f5a1d13d42a0c79f68e9acaaa3…	7500
20257	3675f79c7e05de96ccf189c818b84b487cb1aa3f6b80e8…	8800
25625	fb64af615420e06d46a1965f59068b34460fb3cbe70541…	10000

Insight

- Above are the bottom 10 employees of data scientist role earning less than their peers

[51]:
```
df[(df['tier'] == 1) & ((df['years_of_experience'] >= 5) &␣
↪(df['years_of_experience'] <= 7))].sort_values(by='ctc', ascending=False).
↪head(10)[['company_hash', 'email_hash', 'ctc']]
```

[51]:

	company_hash	email_hash	ctc
25231	onvnt onqttn	fb8410d957fcc2e98cc419e9354dbb3acea2309b0898e2…	37600000
1652	mxsmvoptnwgb	89e0bd3c55896b4b09bb31fa4a736dd6c6d9c3622049e0…	37400000
57410	xmb	46dece7d152edae30f51b0ceab430cbc9681fdd0100e72…	36100000
16073	oytrr xzaxv bvqptno rxbxnta	109ea0d2fea7028684f062eccdc53638489ff2e662ea86…	36000000
70443	stzuvwn	351256c3e18d5d9520baa0f6d7060799d2d81b2cc5c44a…	35000000
22238	vwwtznhqt	fe6af782581bf996a3daacc23387526b8f65435d3c7bd4…	34890000
103184	vnnqv	596b127f7d32653e454c6f42bf74f97af8f8e71d32d352…	34000000
11165	ihtoo wgqu	fce813324a073acc850322b38f11b48e3771c6558bc21e…	33500000
29263	xzaxsg vxqrxzt	14df647edb4209bf283db88e2fea7d52f2e321481dcc4e…	33400000
33201	yvuuxton bxzao ntwyzgrgsxto	59216d0595d84ec308c8ab6107a8e071af0aaa11bff208…	33000000

Insight

- Above are the top 10 employees with 5,6 or 7 years of experience earning more than most of the employees in the company

[52]:
```
df.groupby(['company_hash']).agg({'ctc':'max'}).sort_values(by='ctc',␣
↪ascending=False).head(10)
```

[52]:

company_hash	ctc
onvnt onqttn	37600000
mxsmvoptnwgb	37400000
stztojo	37200000
bgngktz ehtr ojontb	36360000
xmb	36100000
ofxssj	36000000
srgmvr ogenfvqt	36000000
jvygg xzw	36000000
zgn vuurxwvmrt	36000000
oytrr xzaxv bvqptno rxbxnta	36000000

Insight

- Above are the top 10 companies based on their CTC

[53]:
```
temp_df = (
    df[['company_hash', 'job_position', 'ctc']]
    .sort_values(['company_hash', 'ctc'], ascending=[True, False])
    .groupby('company_hash')
    .head(2)
    .sort_values('ctc', ascending=False)
)

temp_df['job_position'].value_counts()
```

[53]:

job_position	
backend engineer	9470
fullstack engineer	7209
other	4879
frontend engineer	3610
engineering leadership	3182
…	
researcher	1
senior mobile applications developer androidios	1
frontend developer	1
matlab programmer	1
fullstack web developer	1

Name: count, Length: 255, dtype: int64

Insight

- backend engineer and fullstack engineer are the top 2 job positions in most of the company with high CTC

0.6.5 Data preparation for modelling

[54]:
```
X = df.drop(columns=['company_hash', 'email_hash', 'orgyear', 'job_position',␣
↪'ctc_updated_year'])
```

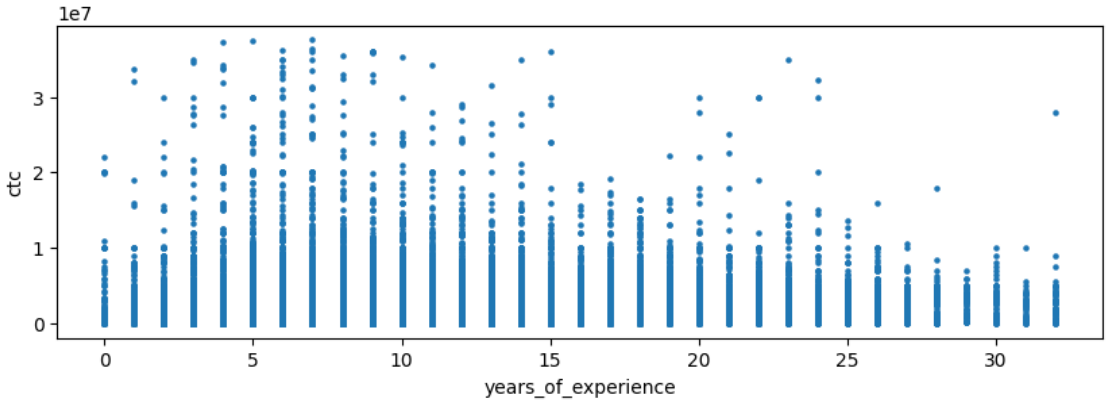
[55]:
```
X.head()
```

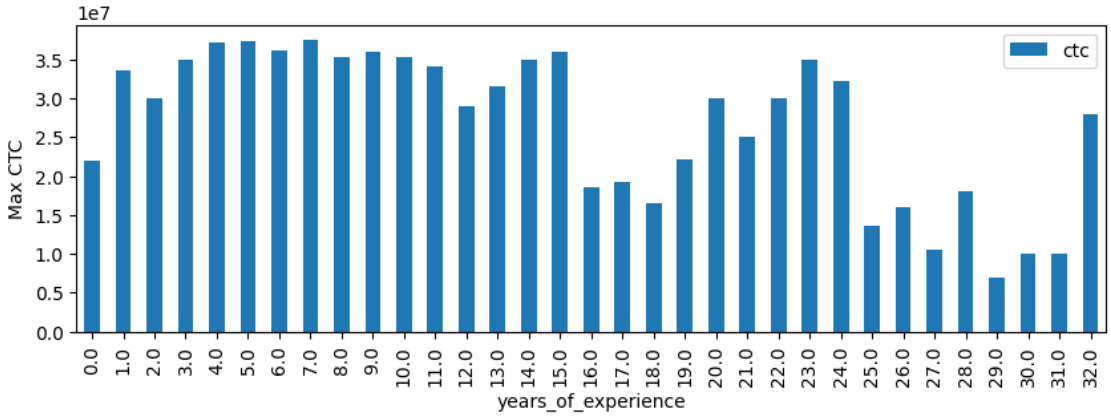
[55]:

	ctc	years_of_experience	years_since_increment	ctc_rnk	designation	\
0	1100000	7.0	3.0	3	2	
1	449999	5.0	4.0	4	3	
2	2000000	8.0	3.0	2	2	
3	700000	6.0	4.0	4	3	
4	1400000	6.0	4.0	1	2	

	class	tier
0	1	2
1	3	3
2	2	2
3	3	3
4	1	1

[56]:
```
X.plot.scatter(x='years_of_experience', y='ctc', s=5, figsize=(10,3))
plt.show()
X.groupby(['years_of_experience']).agg({'ctc':'max'}).plot(kind='bar',␣
↪figsize=(10,3))
plt.ylabel('Max CTC')
plt.show()
```





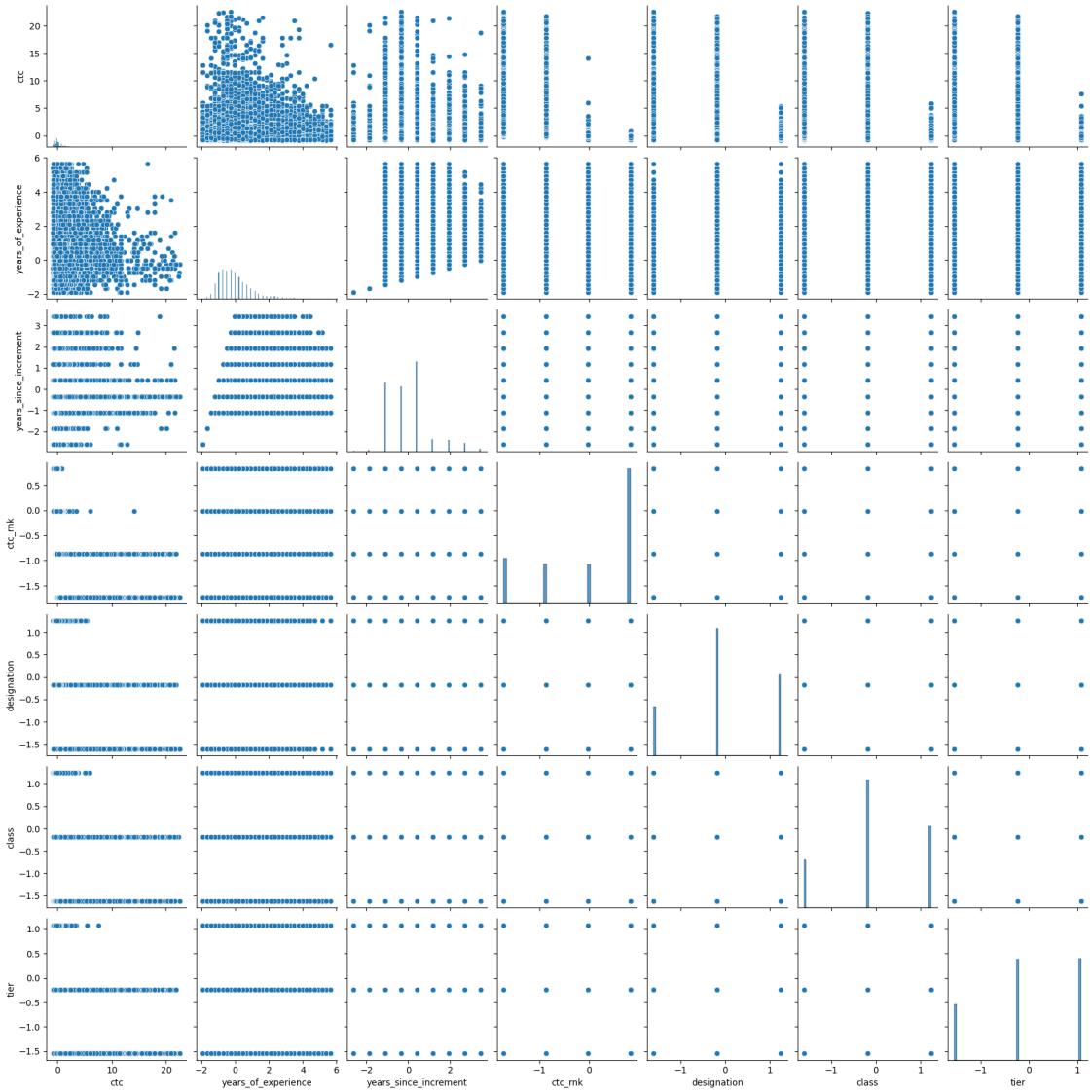
Scale the data

[57]:
```
X_scaled = pd.DataFrame(StandardScaler().fit_transform(X), columns=X.columns)
X_scaled.head()
```

[57]:

	ctc	years_of_experience	years_since_increment	ctc_rnk	designation	class	tier
0	-0.166324	-0.252893	-0.351471	-0.018445	-0.176573	-1.625078	-0.234119
1	-0.570941	-0.723569	0.408135	0.835004	1.252976	1.247282	1.082146
2	0.393913	-0.017555	-0.351471	-0.871894	-0.176573	-0.188898	-0.234119
3	-0.415318	-0.488231	0.408135	0.835004	1.252976	1.247282	1.082146
4	0.020422	-0.488231	0.408135	-1.725343	-0.176573	-1.625078	-1.550384

[58]:
```
sns.pairplot(X_scaled)
plt.show()
```



0.7 Model building

0.7.1 Check Clustering Tendency

[59]:
```
# function to compute hopkins's statistic for the dataframe X
def hopkins_statistic(X):

    X=X.values  #convert dataframe to a numpy array
    sample_size = int(X.shape[0]*0.05) #0.05 (5%) based on paper by Lawson and␣
 ↪Jures

    #a uniform random sample in the original data space
```

```
    X_uniform_random_sample = uniform(X.min(axis=0), X.max(axis=0)␣
 ↪,(sample_size , X.shape[1]))

    #a random sample of size sample_size from the original data X
    random_indices=sample(range(0, X.shape[0], 1), sample_size)
    X_sample = X[random_indices]

    #initialise unsupervised learner for implementing neighbor searches
    neigh = NearestNeighbors(n_neighbors=2)
    nbrs=neigh.fit(X)

    #u_distances = nearest neighbour distances from uniform random sample
    u_distances , u_indices = nbrs.kneighbors(X_uniform_random_sample ,␣
 ↪n_neighbors=2)
    u_distances = u_distances[: , 0] #distance to the first (nearest) neighbour

    #w_distances = nearest neighbour distances from a sample of points from␣
 ↪original data X
    w_distances , w_indices = nbrs.kneighbors(X_sample , n_neighbors=2)
    #distance to the second nearest neighbour (as the first neighbour will be␣
 ↪the point itself, with distance = 0)
    w_distances = w_distances[: , 1]

    u_sum = np.sum(u_distances)
    w_sum = np.sum(w_distances)

    #compute and return hopkins' statistic
    H = u_sum/ (u_sum + w_sum)
    return H
```

[60]:
```
l = [] #list to hold values for each call
for i in range(5):
    H=hopkins_statistic(X_scaled)
    l.append(H)
#print average value:
np.mean(l)
```

[60]:
```
np.float64(0.987559497384396)
```

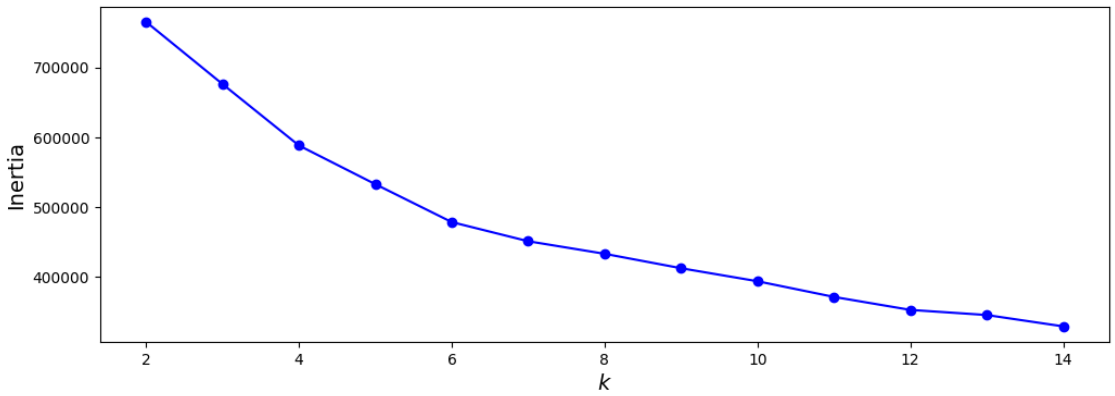
As per Hopkins statistics, with the value of ~0.98, the dataset exhibits very good clustering tendency

0.7.2 Selecting Optimal Number of Clusters

Using the KMeans inertia and Elbow Method

[61]:
```
min_num_of_clusters = 2
max_num_of_clusters = 15
kmeans_per_k = [KMeans(n_clusters=k, random_state=42).fit(X_scaled)
                for k in range(min_num_of_clusters, max_num_of_clusters)]
```

[62]:
```
inertias = [model.inertia_ for model in kmeans_per_k]
#print(inertias)
plt.figure(figsize=(12, 4))
plt.plot(range(min_num_of_clusters, max_num_of_clusters, 1), inertias, "bo-")
plt.xlabel("$k$", fontsize=14)
plt.ylabel("Inertia", fontsize=14)
plt.show()
```



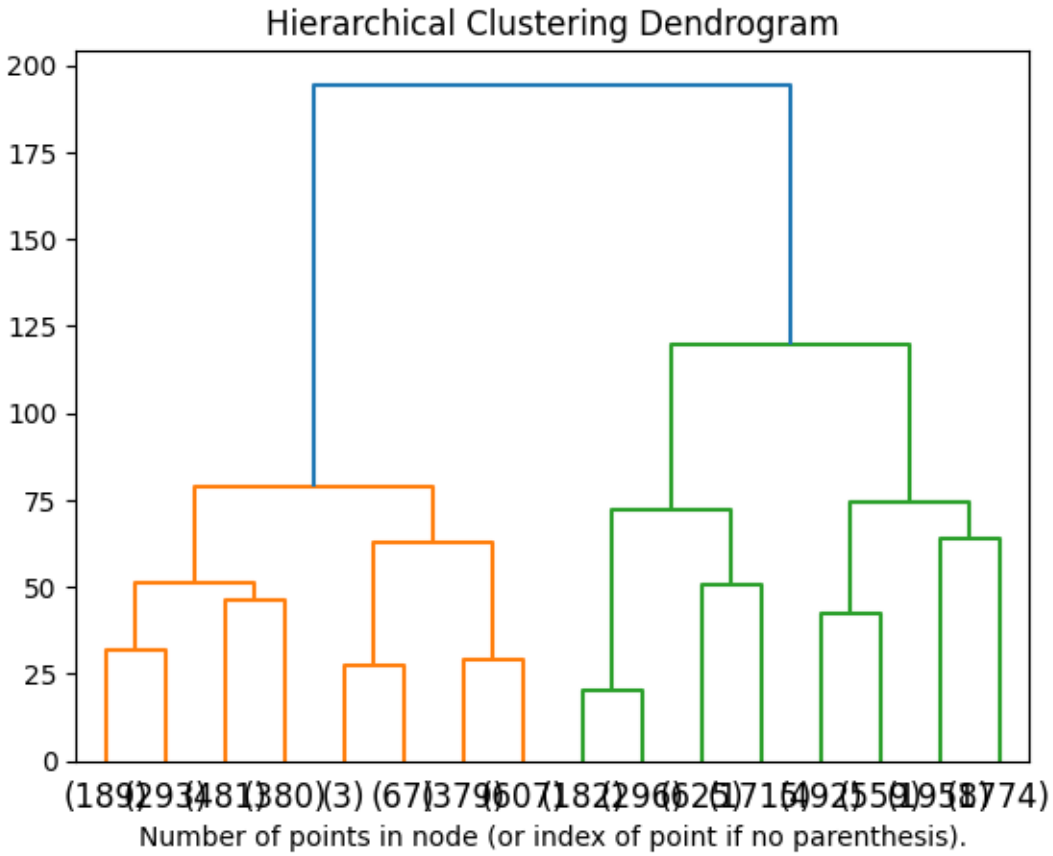
As per the above WCSS, an elbow is created at value 4 and also at 7

0.7.3 Using the Hierarchical Clustering Dendrogram

[63]:
```
def plot_dendrogram(model, **kwargs):
    # Create linkage matrix and then plot the dendrogram

    # create the counts of samples under each node
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1  # leaf node
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count

    linkage_matrix = np.column_stack(
        [model.children_, model.distances_, counts]
    ).astype(float)

    # Plot the corresponding dendrogram
    dendrogram(linkage_matrix, **kwargs)
```

[64]:
```
X_scaled_sample = X_scaled.sample(10000)
X_sample = X.iloc[X_scaled_sample.index]
model = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
model = model.fit(X_scaled_sample)
```

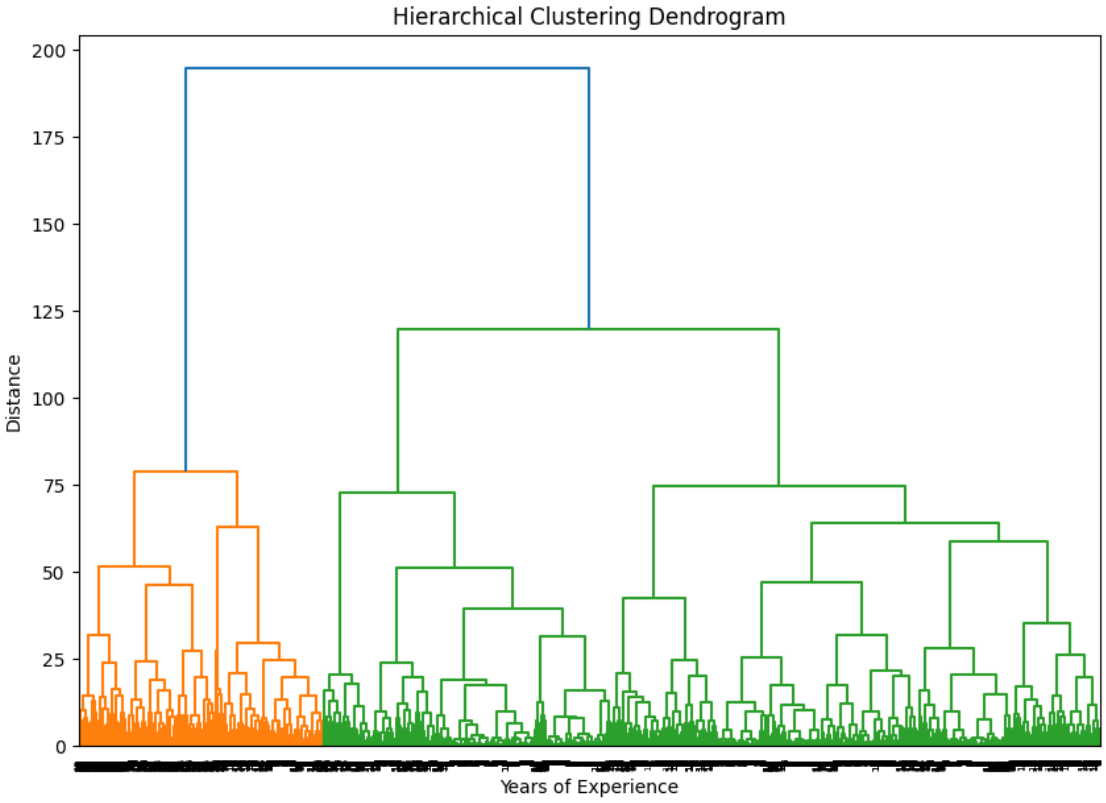
[65]:
```
plt.title("Hierarchical Clustering Dendrogram")
# plot the top three levels of the dendrogram
plot_dendrogram(model, truncate_mode="level", p=3)
plt.xlabel("Number of points in node (or index of point if no parenthesis).")
plt.show()
```



As per the dendrogram plot, number of clusters could be 4

[66]:
```
linkage_matrix = sch.linkage(X_scaled_sample, method='ward')
plt.figure(figsize=(10, 7))
sch.dendrogram(linkage_matrix, labels=X_sample['years_of_experience'].values.
  ↪astype(int))
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Years of Experience')
plt.ylabel('Distance')
```

```
plt.show()
```



Insight

- From the above we can see a clear pattern that the low number of years of experience are grouped together on the extreme left, medium number of years of experience are grouped together in the middle and high number of years of experience are grouped together at the extreme right

0.7.4 K-means Clustering using the optimal number of clusters

[67]:
```
final_num_clusters = 4
kM = KMeans(n_clusters=final_num_clusters, random_state=42)
y_pred = kM.fit_predict(X_scaled)
clusters = pd.DataFrame(X, columns=X.columns)
clusters['label'] = kM.labels_
```

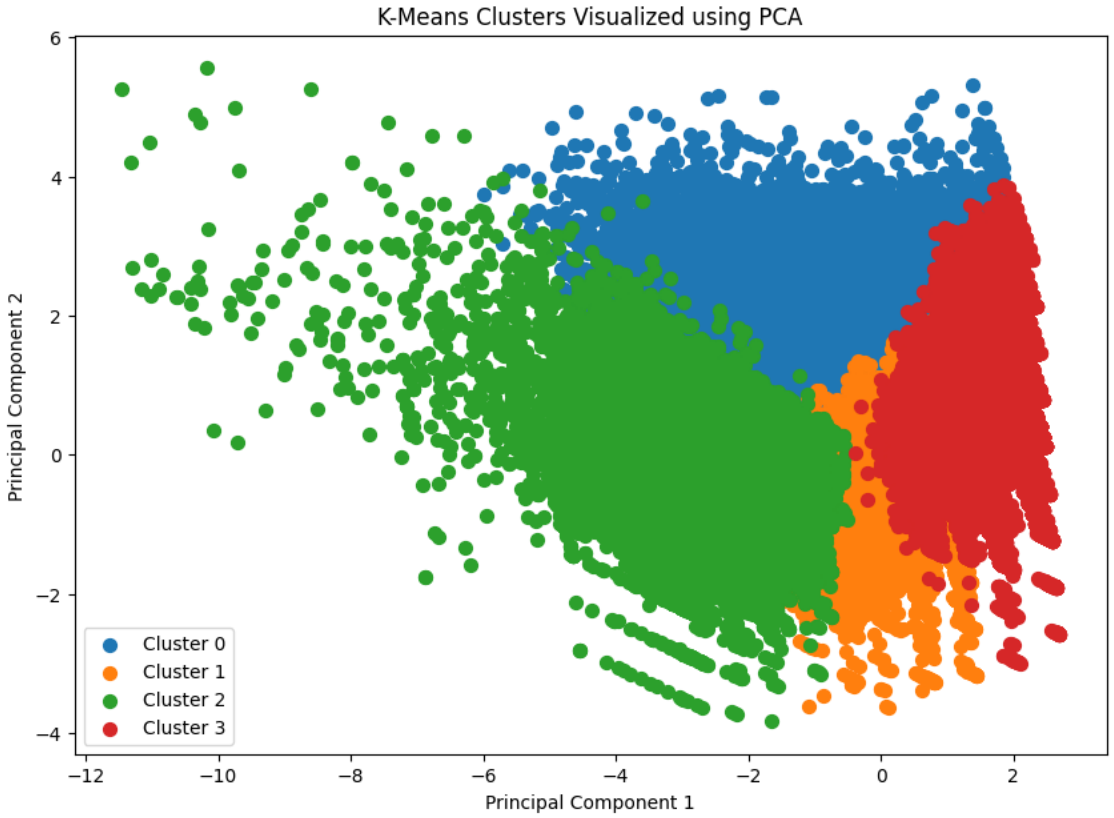
[68]:
```
clusters.head()
```

[68]:
```
      ctc  years_of_experience  years_since_increment  ctc_rnk  designation  \
0  1100000                  7.0                    3.0        3            2
1   449999                  5.0                    4.0        4            3
```

```
2  2000000                 8.0                   3.0        2            2
3   700000                 6.0                   4.0        4            3
4  1400000                 6.0                   4.0        1            2

   class  tier  label
0      1     2      1
1      3     3      3
2      2     2      1
3      3     3      3
4      1     1      2
```

0.7.5 Visualizing clusters using PCA

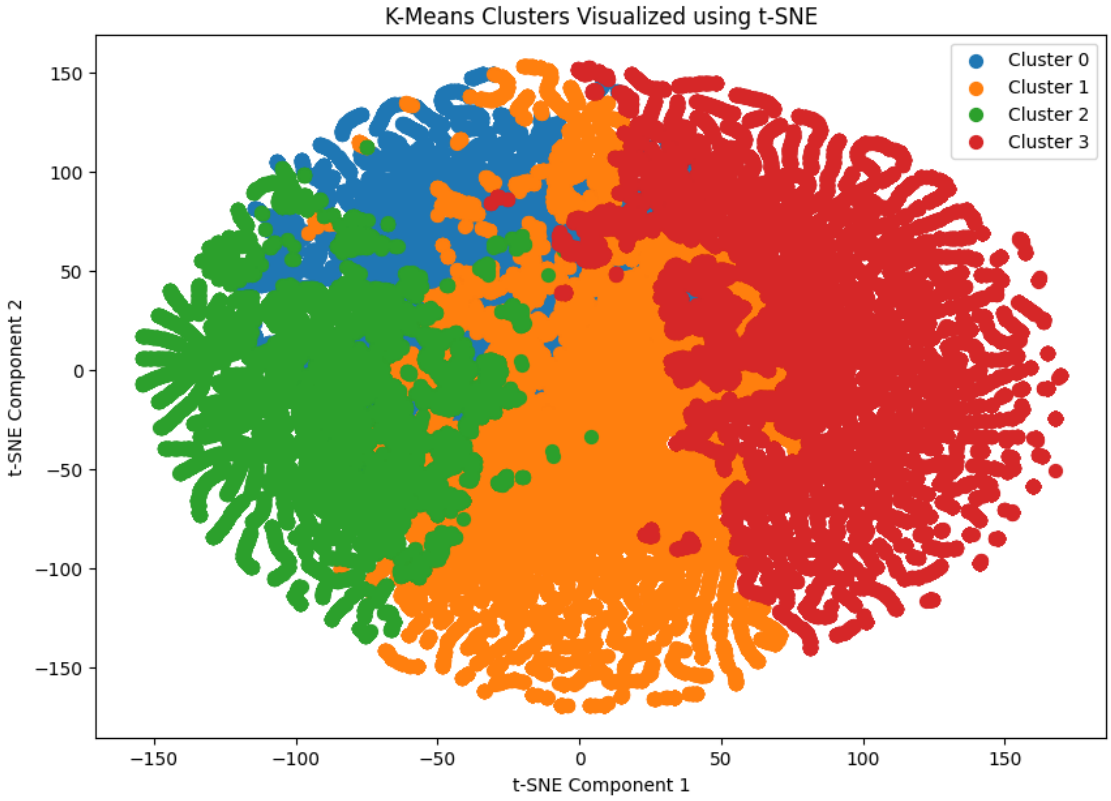
[69]:
```
pca = PCA(n_components=2)
pca_components = pca.fit_transform(X_scaled)
df_pca = pd.DataFrame(data=pca_components, columns=['PC1', 'PC2'])
df_pca['cluster'] = kM.labels_
# Plot the clusters
plt.figure(figsize=(10, 7))
for cluster in range(final_num_clusters):
    clustered_data = df_pca[df_pca['cluster'] == cluster]
    plt.scatter(clustered_data['PC1'], clustered_data['PC2'], label=f'Cluster␣
 ↪{cluster}', s=50)

plt.title('K-Means Clusters Visualized using PCA')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.legend()
plt.show()
```



0.7.6 Visualizing clusters using TSNE

[70]:
```
# Perform t-SNE to reduce to 2 components
tsne = TSNE(n_components=2, random_state=42)
tsne_components = tsne.fit_transform(X_scaled)

# Create a DataFrame with the t-SNE components and cluster labels
df_tsne = pd.DataFrame(data=tsne_components, columns=['TSNE1', 'TSNE2'])
df_tsne['cluster'] = kM.labels_

# Plot the clusters
plt.figure(figsize=(10, 7))
for cluster in range(final_num_clusters):
    clustered_data = df_tsne[df_tsne['cluster'] == cluster]
    plt.scatter(clustered_data['TSNE1'], clustered_data['TSNE2'],␣
 ↪label=f'Cluster {cluster}', s=50)

plt.title('K-Means Clusters Visualized using t-SNE')
plt.xlabel('t-SNE Component 1')
plt.ylabel('t-SNE Component 2')
plt.legend()
```

```
plt.show()
```

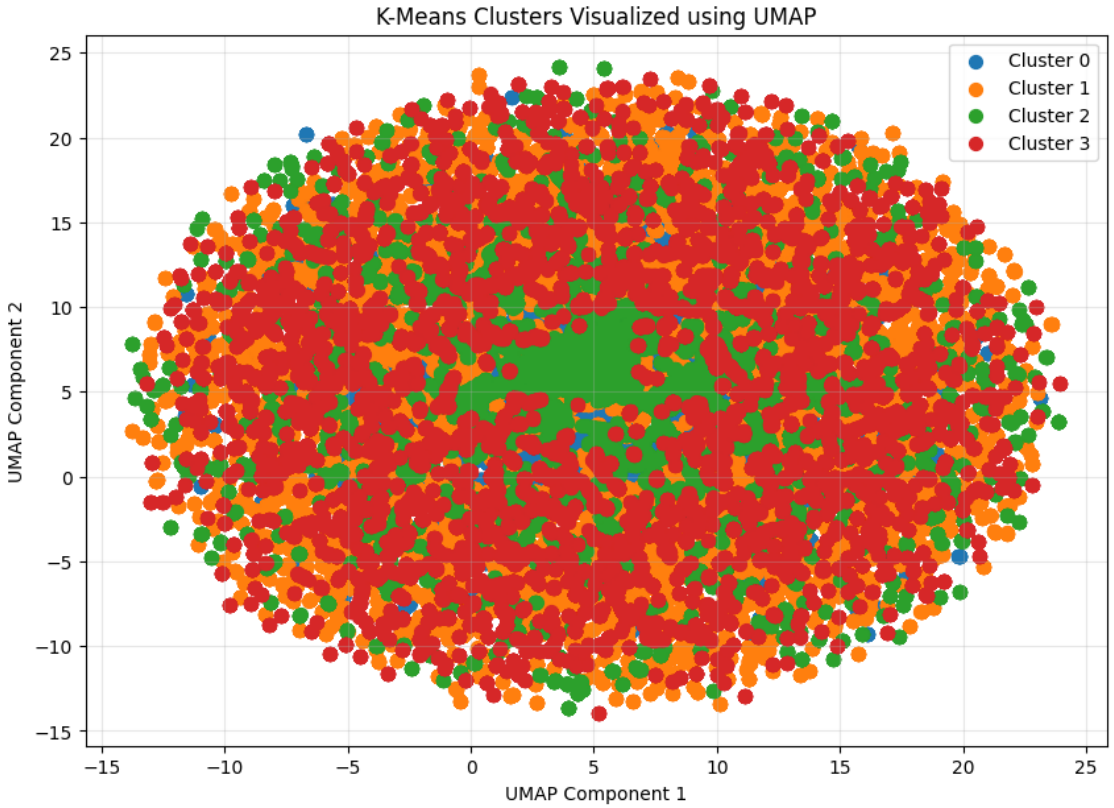


0.7.7 Visualizing clusters using UMAP

[71]:
```
# Add tiny noise to avoid spectral init warning
X_scaled_noisy = X_scaled + np.random.normal(0, 1e-6, X_scaled.shape)

# Perform UMAP to reduce to 2 components
umap_reducer = umap.UMAP(
    n_components=2,
    init='random',        # avoids spectral warning
    random_state=42
)

umap_components = umap_reducer.fit_transform(X_scaled_noisy)

# Create a DataFrame with the UMAP components and cluster labels
df_umap = pd.DataFrame(
    data=umap_components,
    columns=['UMAP1', 'UMAP2']
)
```

```
df_umap['cluster'] = kM.labels_

# Plot the clusters
plt.figure(figsize=(10, 7))

for cluster in range(final_num_clusters):
    clustered_data = df_umap[df_umap['cluster'] == cluster]
    plt.scatter(
        clustered_data['UMAP1'],
        clustered_data['UMAP2'],
        label=f'Cluster {cluster}',
        s=50
    )

plt.title('K-Means Clusters Visualized using UMAP')
plt.xlabel('UMAP Component 1')
plt.ylabel('UMAP Component 2')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
C:\Users\Dell\AppData\Local\Programs\Python\Python313\Lib\site-
packages\umap\umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
```



PCA does a better job at visualizing the clusters in my dataset

[72]:
```
clusters.head()
```

[72]:

	ctc	years_of_experience	years_since_increment	ctc_rnk	designation	class	tier	label
0	1100000	7.0	3.0	3	2	1	2	1
1	449999	5.0	4.0	4	3	3	3	3
2	2000000	8.0	3.0	2	2	2	2	1
3	700000	6.0	4.0	4	3	3	3	3
4	1400000	6.0	4.0	1	2	1	1	2

[73]:
```
unique_labels = clusters['label'].nunique()
colors = sns.color_palette("tab10")[:unique_labels]
```

[74]:
```
clusters.groupby(['label']).agg({'ctc':'mean',
                                 'years_of_experience':'mean',
                                 'years_since_increment':'mean',
                                 'ctc_rnk':pd.Series.mode,
                                 'designation':pd.Series.mode,
                                 'class':pd.Series.mode,
                                 'tier':pd.Series.mode,}).reset_index()
```

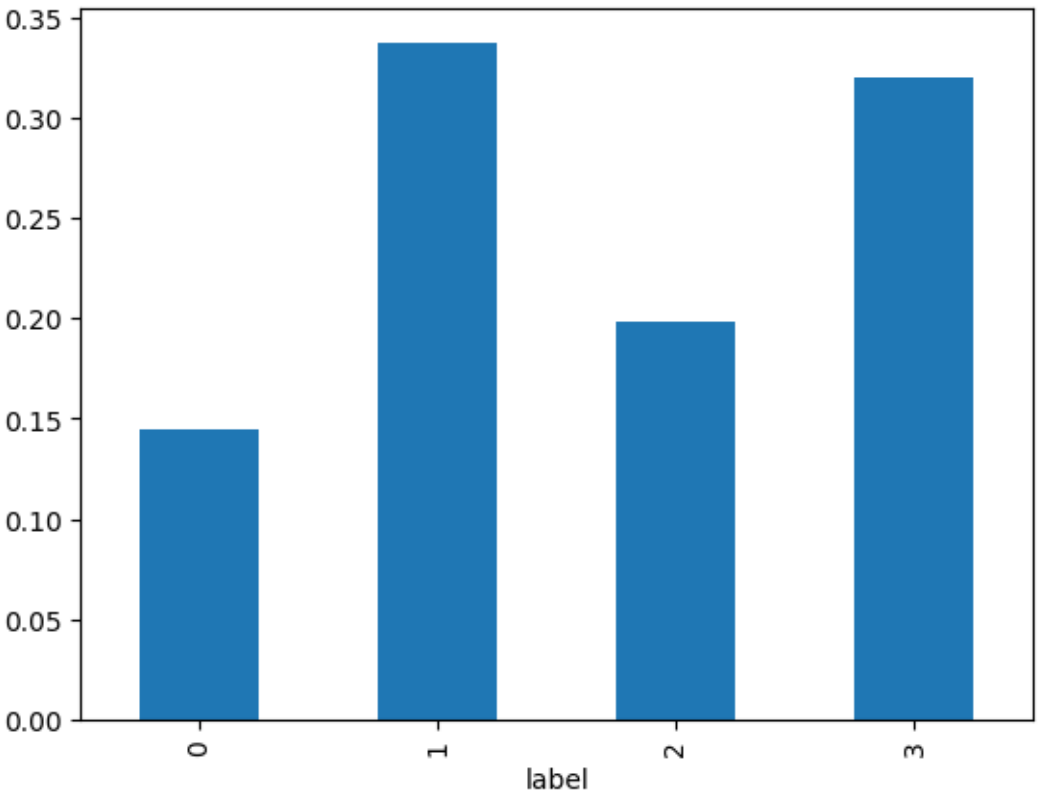
[74]:

	label	ctc	years_of_experience	years_since_increment	ctc_rnk	\
0	0	2.003604e+06	14.562965	4.591388	2	
1	1	8.231663e+05	6.732832	3.168725	4	
2	2	2.871049e+06	8.060462	3.097089	1	
3	3	7.239620e+05	6.573576	3.490035	4	

	designation	class	tier
0	2	2	2
1	2	2	2
2	1	1	1
3	3	3	3

[75]:
```
temp_df = clusters.groupby(['label'])['label'].value_counts()/clusters.shape[0]
print(temp_df)
temp_df.plot(kind='bar')
plt.show()
```

```
label
0    0.144302
1    0.337566
2    0.197984
3    0.320148
Name: count, dtype: float64
```



0.7.8 Insights

- **Optimal Cluster Formation**
  - Four distinct clusters were formed using the Elbow Method and validated through a Dendrogram, indicating well-separated and meaningful learner segments based on experience, compensation, and job attributes.
- **Cluster 3 – Senior High-Earning Segment**
  - Highest mean CTC and highest CTC rank
  - Highest designation level, class, and tier
  - Second highest years of experience
  - Indicates that compensation in this cluster is strongly influenced by role seniority, organizational tier, and responsibility, not just years of experience
  - Represents senior professionals in premium or leadership roles

- **Cluster 0 – Highly Experienced Professionals**
  - Highest years of experience among all clusters
  - Good mean CTC and strong CTC rank
  - High designation, class, and tier
  - Suggests steady career progression where experience is the primary driver of compensation growth
  - Represents experienced professionals in stable, well-paying roles
- **Cluster 2 – Entry-Level Segment**
  - Lowest mean CTC, CTC rank, designation, class, and tier
  - Lowest years of experience
  - Represents early-career professionals or freshers
  - Indicates limited exposure to high-paying roles and senior responsibilities
- **Cluster 1 – Early to Mid-Career Growth Segment**
  - Slightly better CTC, designation, class, and tier compared to Cluster 2
  - Represents professionals transitioning from entry-level to mid-senior roles
  - Shows clear potential for career growth through upskilling and role advancement
- **Overall Career Progression Trend**
  - A clear progression path is visible across clusters: Cluster 2 $\rightarrow$ Cluster 1 $\rightarrow$ Cluster 0 $\rightarrow$ Cluster 3
  - Compensation increases with experience initially, and later with role seniority and organizational tier
- **Learner Distribution Insight**
  - Majority of learners belong to Cluster 1, followed by Cluster 2
  - Indicates that most learners are in early to mid-career stages, actively seeking growth opportunities
  - Senior, high-earning professionals (Clusters 0 and 3) form a smaller but high-value segment

0.7.9 Recommendations

- **Design Career-Progression Focused Courses** Since most learners fall into junior and mid-senior categories (Clusters 1 and 2), Scaler should focus on courses that help learners transition into senior and high-paying roles, emphasizing advanced skills, system design, real-world projects, and interview readiness.

- **Promote High-Salary Career Outcomes** Scaler can attract more learners by highlighting salary potential and growth stories in roles such as Software Developer, Data Analyst, and Data Scientist through targeted ads, testimonials, and outcome-based marketing.

- **Target Academia to Expand Learner Base** Scaler should actively engage with students and educators by collaborating with colleges and universities. This will help increase the learner base while making students industry-ready and employable with competitive salaries.

- **Improve Job Role Data Collection** Learners should be encouraged to specify their job roles more precisely instead of selecting generic categories like "Others." This will lead to better feature quality, improved clustering accuracy, and more actionable insights.

- **Collect Domain-Specific Information** Asking learners to mention their domain of study or work (e.g., software, data, finance, healthcare) will further improve clustering performance and enable domain-specific learning recommendations.

- **Use Clusters for Personalization** The identified clusters can be leveraged to provide personalized learning paths, targeted course recommendations, and role-specific guidance, improving learner engagement and success rates.